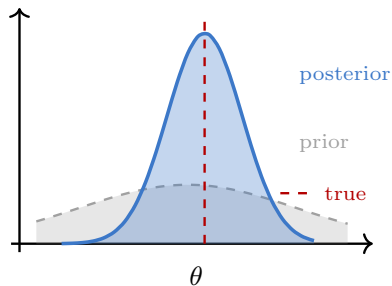


InferCell: Inference-First Whole-Cell Modelling

Tom Kimpson

Department of Mathematics & Statistics,
University of Melbourne

MACSYS UoM Seminar, April 2026



Takeaways

1. **InferCell** is a modular, inference-first whole-cell modelling framework in Julia.
2. Interested? Get involved!

Contents

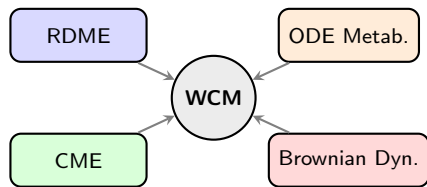
1. The problem
2. InferCell
3. How it works
4. Technical details
5. In practice

The Problem

Why whole-cell models need inference

Whole-Cell Models: The State of the Art

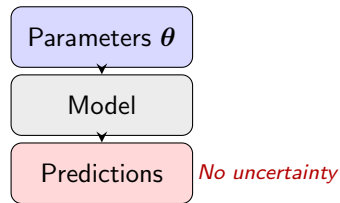
- Whole-cell models simulate a living cell from its molecular parts
- State of the art: Luthey-Schulten Lab's 4D model (MC4D, *Cell* 2026)
 - 493 genes, 4 coupled solvers
 - 3D spatial resolution at 10 nm voxels
 - GPU-accelerated, full cell cycle
- A genuine scientific achievement!



*4 solvers, 3 languages
coupled via shared dictionary*

But It's a Forward Simulator

- ~ 2000 fixed parameters θ
- One prediction out
- This is the norm across the field



The Inference Gap

1. Parameters are uncertain.

Rate constants from different organisms, conditions, experiments. Typical uncertainty: 10–50%+.

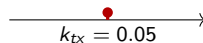
2. Uncertainty propagates nonlinearly.

Uncertain rates $\rightarrow$ mRNA $\rightarrow$ protein $\rightarrow$ flux $\rightarrow$ growth rate.

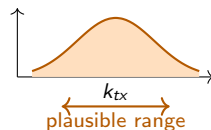
3. The architecture has no place to put it.

MC4D stores parameters as names $\rightarrow$ floats.
No distributions, no uncertainty.

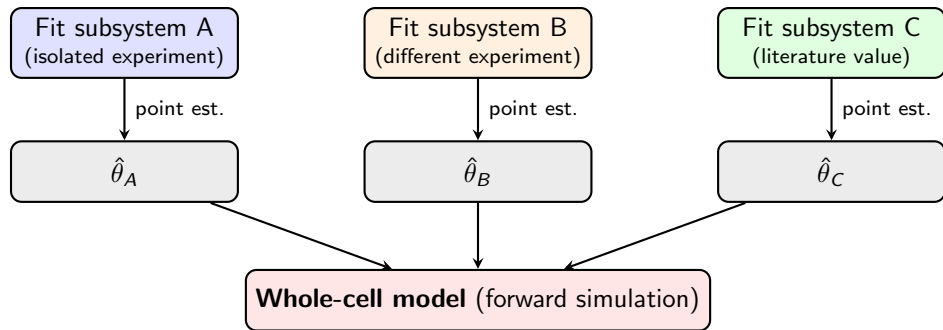
What we pretend



What we actually know



The Standard Workflow

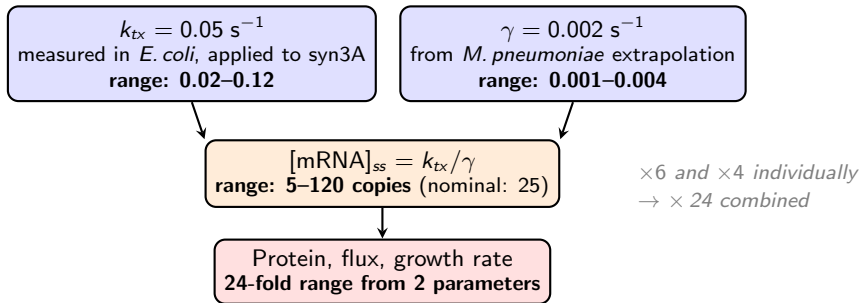


- Each subsystem calibrated in isolation — coupling is invisible during fitting
- Uncertainty discarded at every arrow
- Parameters that are jointly plausible in the coupled model may never be found

What Does That Uncertainty Look Like?

A minimal model with two parameters:

$$\frac{d[\text{mRNA}]}{dt} = k_{tx} - \gamma \cdot [\text{mRNA}] \Rightarrow [\text{mRNA}]_{ss} = k_{tx}/\gamma$$



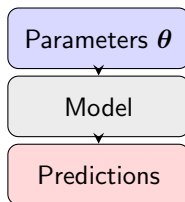
Two parameters. One equation. Twenty-four-fold uncertainty in predictions.
MC4D has **~2000 parameters** feeding coupled nonlinear dynamics.

InferCell

An inference-first approach

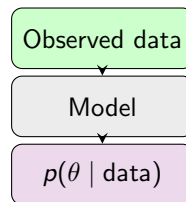
Inverting the Question

Forward simulation



"Given θ , what does the cell do?"

Inference



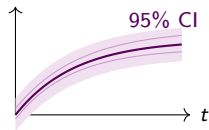
"Given data, what θ are consistent?"

What Inference Enables

Inference lets you ask scientific questions that forward simulation cannot pose.

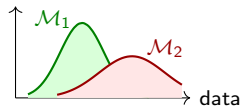
1. Are my predictions trustworthy?

Propagate parameter uncertainty to get credible intervals on outputs. Robust predictions vs. fragile ones.



2. What biology can I learn?

Bayesian model selection. Posteriors on shared parameters reveal how subsystems interact.



3. What can I learn about my model?

Which parameters are identifiable? Does the formalism matter (ODE vs SSA)? Where does the model fail to reproduce data? What experiment should you run next?



identifiable $\Rightarrow$ data-informative

InferCell: Design Principles

- **Inference-first.**

UQ and probabilistic prediction built in, not bolted on.

- **Modular.**

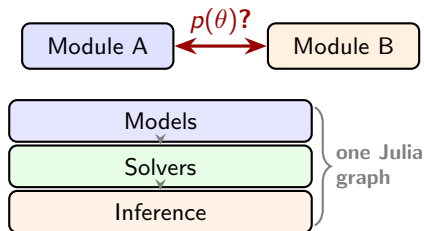
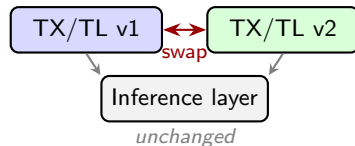
Swap modules; inference still works.

- **Information flow as design test.**

Every boundary: “how does inference cross this?”

- **Pure Julia.**

One language, one computational graph, no barriers to AD.



How It Works

Architecture and inference graph

One Graph, End to End

Probabilistic prediction requires gradients end-to-end.

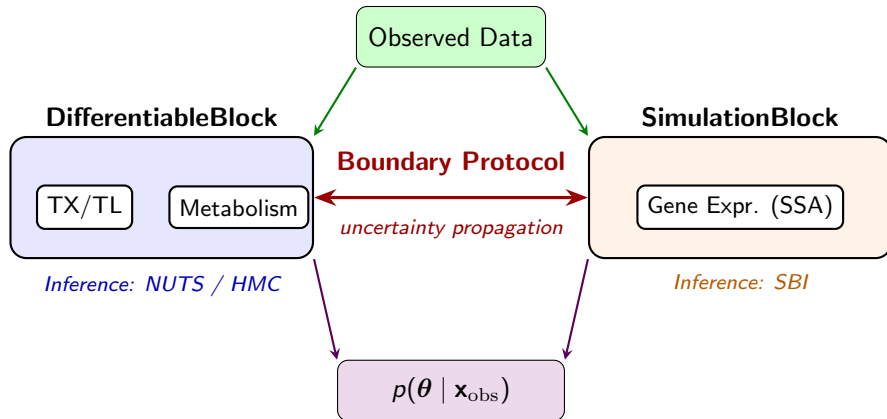
- MC4D: Python + Cython + CUDA + LAMMPS — no gradient path through language boundaries
- InferCell: solvers, models, and inference share **one compiled computational graph** in Julia

Julia ecosystem:

- DifferentialEquations.jl — solvers
- Turing.jl — Bayesian inference
- SciMLSensitivity.jl — AD through solvers
- Catalyst.jl — reaction networks
- Native interop between all of these

AD, likelihood evaluation, and MCMC sampling fall out of this design.

The Inference Graph



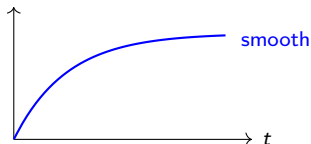
Modules are automatically partitioned into blocks by their declared inference mode.

Why Hybrid Inference?

Cells mix fundamentally different dynamics in the same model:

Differentiable dynamics

ODE metabolism, continuous kinetics

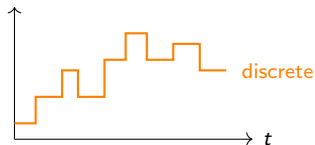


- Gradients exist
- **NUTS / HMC**

Orders of magnitude more efficient

Stochastic dynamics

Gillespie SSA, gene switching



- No gradient path
- **SBI / ABC-SMC**

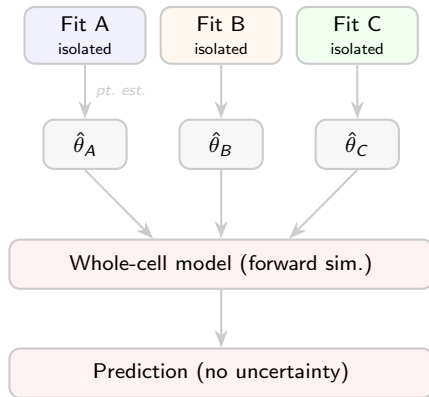
Handles any simulator as black box

No single inference method handles both.

The hybrid graph uses each method where it is strongest.

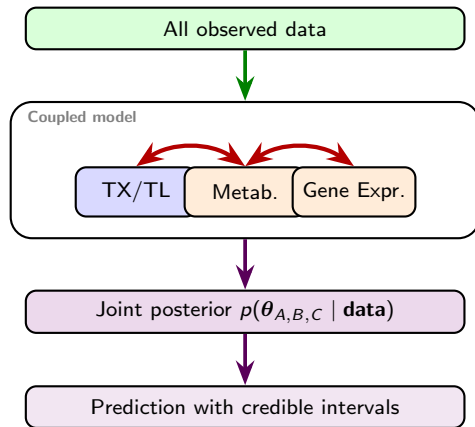
Conditioning the Whole Model

Standard approach



- ✗ uncertainty discarded at every step
- ✗ coupling invisible during fitting

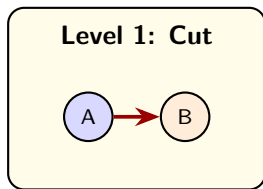
InferCell approach



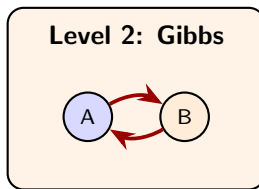
- ✓ full uncertainty propagated throughout
- ✓ coupling visible during inference

How Much Coupling Do You Need?

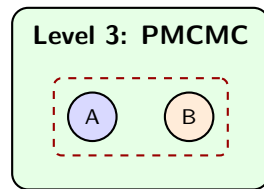
Joint inference over all parameters is intractable.
Isolated submodel fits discard all coupling.



Downstream
sees upstream
uncertainty; no feedback



Information flows both
ways; approximate



Asymptotically correct;
expensive

isolated fits

accuracy / cost $\longrightarrow$

full joint

Technical Details

Module interface and parameter specification

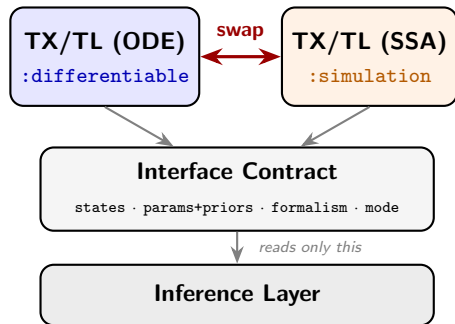
The Interface Is the Architecture

The module internals are swappable by design. What makes inference compose is the **boundary contract**:

- **States** — typed variables in, typed variables out
- **Parameters** — each carries a prior distribution
- **Formalism** — `:ode`, `:sde`, `:jump`
- **Inference mode** — `:differentiable` or `:simulation`

The inference layer sees *only* the interface. It never looks inside the module.

Design test: can you swap the internals without changing the inference call?



Modules Declare Their Inference Mode

Each module is defined by a **typed interface** that declares its dynamics *and* how inference should work:

```
abstract type AbstractSubModel end

states(m) # state variables
parameters(m) # with priors
dynamics(u, p, t, m) # rate equations
formalism(m) # :ode, :sde, :jump
inference_mode(m) # :differentiable,
                  # :simulation, :auto
```

- The framework reads these declarations and **automatically partitions** modules into the right inference backend
- Adding a module extends the inference problem **without code changes** to the inference layer
- Swap an ODE model for an SSA model — the backend switches automatically

Modularity at the inference level, not just the simulation level.

Parameters Carry Priors

Every parameter declares its value, prior distribution, and whether it is free or fixed:

```
struct InferParameter
  value::Float64
  prior::Distribution
  fixed::Bool
  name::Symbol
  role::Symbol # :rate, :initial,
               # :observation
end
```

- **Fixed:** point estimate from literature
- **Constrained:** informative prior from experiment
- **Free:** to be inferred from data
- The inference layer reads these declarations and builds the Bayesian model automatically
- Sensitivity analysis is a flag toggle, not a code change

End-to-End Workflow

```
using InferCell

# 1. Define sub-model
txl = TranscriptionTranslation()

# 2. Build problem (orchestrator composes modules)
prob = build_problem([txl]; tspan=(0.0, 100.0))

# 3. Generate synthetic data
true_sol = solve(prob, Tsit5())
data = observe(true_sol, 0.0:5.0:100.0; sigma=0.1)

# 4. Infer parameters (NUTS via Turing.jl)
chain = infer(txl, data)

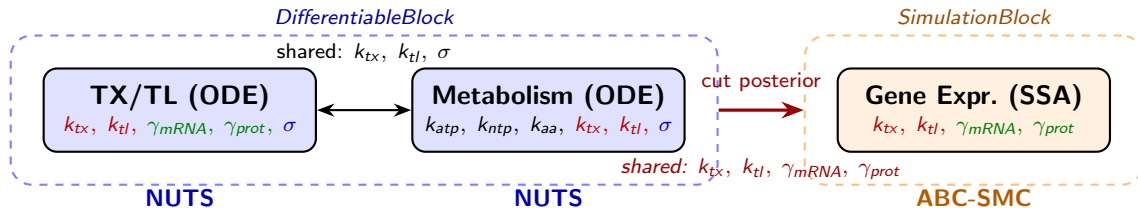
# 5. Posterior predictive check
ppc = posterior_predictive(txl, chain)
```

Same `infer()` call works for a single module or a composed multi-module model.

In Practice

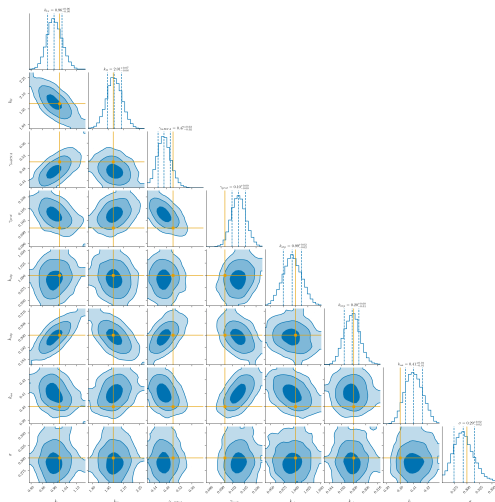
v1 scope and preliminary results

v1 Scope: Three Modules



- Same biology (TX/TL) modelled as ODE and SSA $\rightarrow$ direct comparison of posteriors across formalisms
- ~ 9 unique parameters after deduplication (shared across modules), all with priors
- Validated on synthetic data where ground truth is known

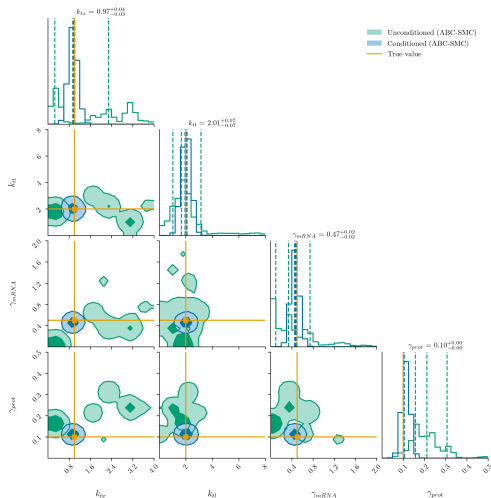
Parameter Recovery: Joint ODE Inference (TX/TL + Metabolism)



Joint posterior over 9 parameters

- TX/TL and Metabolism modules inferred jointly via NUTS
- Yellow crosshairs: ground truth values
- Posteriors recover the true parameters
- Off-diagonal panels reveal parameter correlations

Preliminary Results: SSA Module



Parameter recovery: gene expression (SSA)

- 4 parameters inferred via ABC-SMC
- Green: unconditioned posteriors
- Blue: conditioned on shared parameters from the ODE module
- Conditioning tightens the posteriors. Information flows between modules

Summary

1. **The problem.** Whole-cell models take ~ 1000 parameters from literature and simulate forward. No uncertainty quantification, no way to learn from data, no way to know which predictions to trust.
2. **InferCell inverts the question.** Given observations, infer parameters and their uncertainties. The model becomes something you condition on data, not just run forward.
3. **Joint inference over the coupled model.** Modules are composed into a hybrid inference graph with tuneable coupling from cut posteriors to exact joint inference.

Takeaways

1. **InferCell** is a modular, inference-first whole-cell modelling framework in Julia.
2. Interested? Get involved!
 - github.com/MACSYS-CoE/InferCell
 - Please come talk to me or [open an issue](#)
 - Workshop session at QUT in May (? TBC)

Backup

Boundary Protocol: Three Levels

1. **Level 1 — Sequential conditioning (v1 target)**

Infer differentiable block $\rightarrow$ posterior samples $\rightarrow$ condition simulation block on those samples.

One-directional. Produces a cut posterior. Valid under conditional independence.

2. **Level 2 — Gibbs-like alternation (v1 demonstration)**

Alternate: fix stochastic module state $\rightarrow$ NUTS on differentiable params; fix differentiable params $\rightarrow$ SBI on stochastic params.

Bidirectional. Pseudo-marginal Gibbs sampler. Convergence depends on ABC tolerance.

3. **Level 3 — Particle MCMC (future work)**

Stochastic modules contribute unbiased likelihood estimates via particle filters into a joint MCMC sampler.

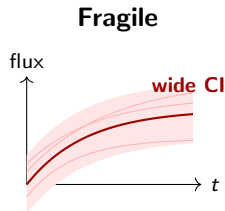
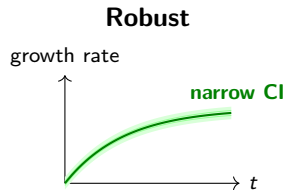
Asymptotically exact. Computationally expensive.

Which Predictions Can You Trust?

Propagate the posterior through the model.
Some predictions are **robust**, others are **fragile**.

- **Robust prediction:**
Barely changes across the posterior. You can trust this output regardless of parameter uncertainty.
- **Fragile prediction:**
Swings wildly. Sensitive to parameters the data cannot constrain. This is where you need more data — or a simpler model.

Forward simulation never tells you which is which.
Every prediction looks equally confident.

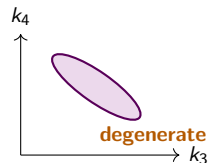
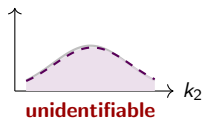
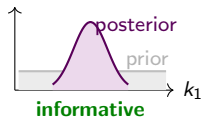


What Your Model Actually Depends On

With point estimates, every parameter looks equally important.

With posteriors, you learn **which parameters matter**:

- **Tight posterior** (narrower than prior):
Data constrains this parameter. It matters.
- **Posterior $\approx$ prior**:
Data says nothing. Redundant, or not observable.
- **Correlated posteriors**:
Parameters trade off. Neither is identifiable alone, but their combination is constrained.



These are scientific insights — they tell you what to measure next.
Forward simulation never reveals them.

TX/TL Module: Equations

Two-state ODE with four rate parameters:

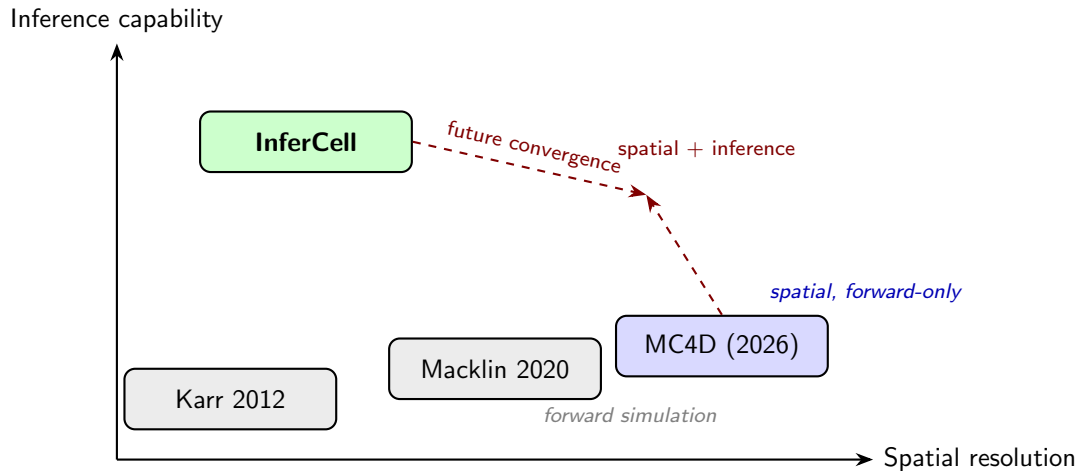
$$\frac{d[\text{mRNA}]}{dt} = k_{tx} - \gamma_{mRNA} \cdot [\text{mRNA}]$$

$$\frac{d[\text{protein}]}{dt} = k_{tl} \cdot [\text{mRNA}] - \gamma_{prot} \cdot [\text{protein}]$$

Parameter	Description	Prior
k_{tx}	Transcription rate	$\text{LogNormal}(-3.0, 0.5)$
k_{tl}	Translation rate	$\text{LogNormal}(-1.5, 0.5)$
γ_{mRNA}	mRNA degradation rate	$\text{LogNormal}(-4.0, 0.5)$
γ_{prot}	Protein degradation rate	$\text{LogNormal}(-5.0, 0.5)$
σ	Observation noise	$\text{HalfNormal}(0.5)$

Observation noise σ is inferred, not fixed.

Complementary to Existing Work



Beyond Parameter Uncertainty: Which Reactions Exist?

So far: **fixed model structure**, infer rate constants.

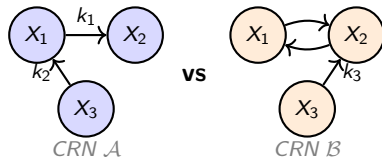
But often the structure itself is uncertain:

- Is gene X regulated by protein Y ?
- Does this metabolite inhibit that enzyme?
- Which of several proposed pathways is active?

Different CRN structures can fit training data equally well but **diverge catastrophically under novel conditions** (Foo et al. 2025, Fig. 2: reconstruction error small, prediction error $10\times$ larger for wrong structure).

Structural uncertainty can dominate parameter uncertainty.

Which CRN?



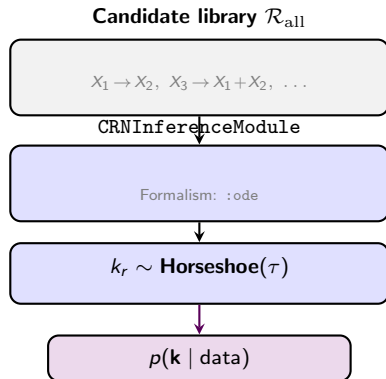
Both fit the data.
Only one predicts correctly
under new conditions.

CRN Inference as a Plug-and-Play Module

Key idea: include *all* candidate reactions in one module. Let the **posterior** decide which are present.

- Define a candidate library $\mathcal{R}_{\text{all}}$
- Dynamics: $\dot{x}_s = \sum_r \nu_{sr} k_r \prod_j x_j^{m_{rj}}$
- One rate constant k_r per reaction
- Horseshoe prior: $k_r \sim \text{HS}(\tau)$
Pushes most $k_r \rightarrow 0$
- Posterior over $\mathbf{k}$ is the posterior over structures

No new inference machinery needed.
Same `infer()` call. The prior does the work.



How the Horseshoe Prior Induces Structure

The **horseshoe prior** $k_r \mid \lambda_r \sim \mathcal{N}^+(0, \lambda_r^2 \tau^2)$, $\lambda_r \sim \text{C}^+(0, 1)$ has two key properties:

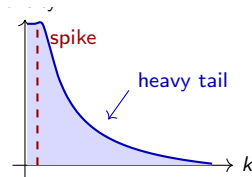
1. **Spike at zero.** Most rate constants are shrunk to effectively zero $\rightarrow$ those reactions are “absent.”
2. **Heavy tails.** Reactions that *are* present retain their full magnitude — minimal bias on the non-zero rates.

The posterior naturally separates into:

- $k_r \approx 0$: reaction r absent
- $k_r \gg 0$: reaction r present

Reaction inclusion probability:

$$P(r \in \mathcal{R} \mid \text{data}) = P(k_r > \epsilon \mid \text{data})$$



Posterior for 5 candidate reactions

k_1		present
k_2		
k_3		absent
k_4		
k_5		uncertain

Relation to Foo et al. (2025): Structural Uncertainty in CRNs

Their approach:

1. Sparse optimisation (BFGS) with penalty functions (L1, horseshoe, etc.)
2. Multiple restarts $\times$ multiple λ values
3. Keep all local optima $\rightarrow$ map to CRN structures
4. BIC approximation for model evidence
5. Approximate posterior over CRN structures

Key results:

- Recovers dynamically equivalent CRNs
- Identifies structural ambiguities via posterior correlations
- Hierarchical tree of alternative pathways

Foo, Zanca, Flegg & Siekmann (2025). *Quantifying structural uncertainty in chemical reaction network inference*. arXiv:2505.15653.

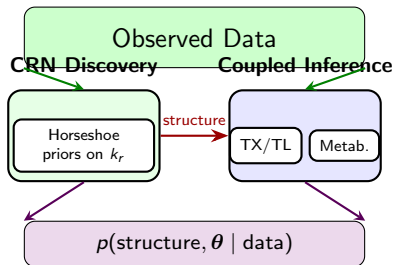
Can InferCell do this natively?

- Penalty functions $\rightarrow$ **horseshoe prior**
Same sparsity, fully Bayesian
- Multi-start optimisation $\rightarrow$ **NUTS sampling**
Explores multimodal posterior directly
- BIC approximation $\rightarrow$ **exact posterior**
No need for separate model evidence step
- Separate pruning/recombination pipeline $\rightarrow$ **not needed**
Posterior samples directly encode structure

Composing Structural Inference with the Inference Graph

Structural inference plugs into the full InferCell architecture:

- **Stage 1:** Discover which reactions are active from data (CRN inference module)
- **Stage 2:** Full Bayesian inference on the discovered structure, composed with other modules
- Or run both simultaneously — horseshoe parameters live alongside standard rate constants in the same NUTS sampler



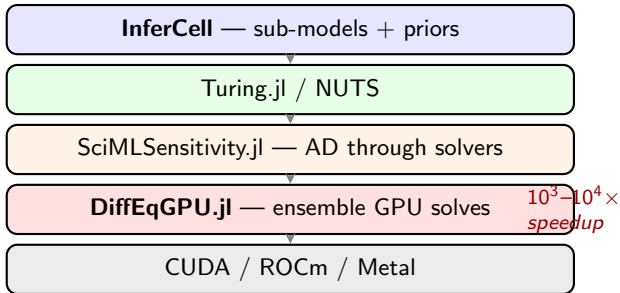
GPU Acceleration and the Julia Advantage

The bottleneck: every NUTS leapfrog step solves the full ODE system. As models grow, this means $O(10^4)$ ODE solves per chain.

Julia's single-language stack:

- **DiffEqGPU.jl:** batch ODE solves on GPU — same code, one-line backend switch
- **AD composes with GPU kernels:** gradients through GPU-accelerated solvers, no custom CUDA
- **KernelAbstractions.jl:** write once, run on CUDA, ROCm, Metal, or CPU

No serialisation between sampler $\leftrightarrow$ solver $\leftrightarrow$ GPU. One compiled representation top to bottom.



What this enables:

- Parallel MCMC chains across GPU cores
- Posterior predictive checks batched on GPU
- Adjoint sensitivity methods scale to 100+ parameters without linear cost growth

Why Julia (and Not JAX)?

Julia wins on composability:

- DiffEq + Turing + SciMLSensitivity + Catalyst share one type system — AD flows end-to-end with no language boundaries
- Multiple dispatch makes the `AbstractSubModel` interface natural: `formalism(m)` dispatches to the right solver
- Catalyst.jl gives a CRN DSL with no JAX equivalent — critical for structural inference

For v1, Julia is OK:

The research contribution is the architecture and boundary protocol, not the scale. Julia lets us express the ideas most directly.

JAX would be ahead on:

- **SBI tooling.** Python `sbi` library is more mature (neural density estimators, normalizing flows)
- **GPU at scale.** XLA compilation, `vmap/pmap` over samples — smoother path to 100+ parameters
- **Community.** Larger pool of computational biology contributors
- **ML integration.** Neural surrogates, amortized inference live in PyTorch/JAX

The risk to watch:

If InferCell needs neural SBI or heavy GPU batching at whole-cell scale, the ecosystem gap could be annoying.